

Lab 05 — Hands-on lab: from JDK to JRE, from Node to nginx

Goal: build the same API twice — in one step, then multi-stage — measure the difference, do the same for an "Angular" front end, then touch build caches and an image without a shell.

Prerequisites — Lab 04 done: `~/labo-docker/04/Api.java` exists, the images `eclipse-temurin:21-jdk` and `eclipse-temurin:21-jre-alpine` are present.

Files provided (`files/`) - `web/package.json` and `web/src/index.html` — a fake Angular project. The "build" will be simulated by a `cp`: we reproduce the **shape** of a front-end project, not its content.

You will write every Dockerfile yourself: that is the exercise.

Step 1 — The "all-in-one" image

```
mkdir -p ~/labo-docker/05 && cd ~/labo-docker/05
cp ~/labo-docker/04/Api.java .
```

Create `Dockerfile.mono` — compilation **and** execution in the same image:

```
FROM docker.io/library/eclipse-temurin:21-jdk
WORKDIR /app
COPY Api.java .
RUN mkdir -p build && javac -d build Api.java && jar --create --file api.jar --main-class Api -C build .
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
```

```
podman build -f Dockerfile.mono -t api-mono:1.0 .
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}' | grep -E 'api-
mono|temurin'
```

Observe `localhost/api-mono 1.0 488 MB` — exactly the size of `eclipse-temurin:21-jdk`. The 2 KB JAR added nothing; the JDK brought everything.

```
podman run --rm --entrypoint sh api-mono:1.0 -c 'ls /app; javac -version'
```

Observe `Api.java`, `api.jar`, `build`, and `javac 21.0.x`: the source code **and** the compiler are in the production image.

Explanation. This image works perfectly. That is what makes it dangerous: nothing signals that 480 MB of tooling and your sources go out with every deployment.

Step 2 — Multi-stage

Create `Dockerfile`:

```
# ----- stage 1: build -----
FROM docker.io/library/eclipse-temurin:21-jdk AS build
WORKDIR /src
COPY Api.java .
RUN mkdir -p build && javac -d build Api.java && jar --create --file api.jar --main-class Api -C build .

# ----- stage 2: runtime -----
FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /src/api.jar /app/api.jar
USER 1000:1000
EXPOSE 8080
ENTRYPOINT ["java","-jar","/app/api.jar"]
```

```
podman build -t api-multi:1.0 .
```

Observe the prefixes `[1/2] STEP 1/4 ...` then `[2/2] STEP 1/6 ...`: Buildah numbers the stages, and only the last one ends with `COMMIT api-multi:1.0`.

```
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}' | grep -E 'api-mono|api-multi'
podman run --rm --entrypoint sh api-multi:1.0 -c 'ls /app; javac -version'
podman run --rm --entrypoint ls api-multi:1.0 /src
```

Observe `209 MB` against `488 MB`, then `api.jar` alone, `sh: javac: not found`, and `ls: cannot access '/src': No such file or directory`.

Explanation. The `build` stage existed for the time of the compilation, then was discarded. The final image only knows the file copied by `COPY --from`. No sources, no JDK, no `/src` folder: they were never part of its layers.

```
podman history api-multi:1.0 --format 'table {{.Size}}\t{{.CreatedBy}}' | head -6
```

Observe a `4.61kB` layer for the `COPY` of the JAR: everything else comes from the base image.

PITFALL

`COPY --from=build /src /app` would have copied the whole folder, `Api.java` and `build/` included. You copy **the artefact**, not the working directory. That is question 3.

Step 3 — Looking inside a stage

A discarded stage is not inspectable... unless you ask to stop there:

```
podman build --target build -t api-build-stage .
podman run --rm api-build-stage ls -la /src
```

Observe `Api.java`, `api.jar`, `build/`: that is the exact content of the stage at the moment stage 2 copied `api.jar` from it.

Explanation. `--target` is the diagnostic tool of multi-stage: when a `COPY --from` fails with `no such file or directory`, you build the stage alone and look, instead of guessing paths (question 12).

```
podman rmi api-build-stage
```

Step 4 — The front end: Node builds, nginx serves

```
mkdir -p web && cp -r <lab-path>/files/web/* web/  
ls -R web
```

Create `web/Dockerfile`:

```
FROM docker.io/library/node:22-alpine AS build  
WORKDIR /app  
COPY package.json .  
RUN echo "npm ci (simulated)"  
COPY src ./src  
RUN mkdir -p dist/web/browser && cp src/index.html dist/web/browser/ && echo "ng build (simulated)"  
  
FROM docker.io/library/nginx:alpine  
COPY --from=build /app/dist/web/browser /usr/share/nginx/html  
EXPOSE 80
```

```
podman build -t web-multi:1.0 web  
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}' | grep -E 'web-multi|node|nginx'
```

Observe `web-multi 1.0 64.2 MB` — exactly the size of `nginx:alpine` — whereas `node:22-alpine` weighs `167 MB`.

```
podman run -d --name web -p 18081:80 web-multi:1.0  
curl -s localhost:18081/  
podman rm -f -t 0 web
```

Observe your `index.html` served by nginx. Also open `http://localhost:18081/` in the Windows browser.

Explanation. Node served to "build" (here, a `cp` stands in for `ng build`), then disappeared. The front end in production is a static file server: that is why the `web` image of an Angular stack never contains Node.

→ ANGULAR

On a real project, replace `RUN echo "npm ci (simulated)"` with `RUN npm ci` and the `cp` with `RUN npm run build`, and copy `dist/<project-name>/browser`. The `COPY package*.json → npm ci → COPY . .` separation is the cache separation of lab 04: the 900 MB of `node_modules` are only re-downloaded if `package-lock.json` changes.

Step 5 — A cache that survives builds

Create `Dockerfile.cache`:

```
FROM docker.io/library/eclipse-temurin:21-jdk AS build
WORKDIR /src
COPY Api.java .
RUN --mount=type=cache,target=/root/.m2 sh -c 'echo "dep-$(date +%s)" >> /root/.m2/marker; cat
/root/.m2/marker' \
&& mkdir -p build && javac -d build Api.java

FROM docker.io/library/alpine
COPY --from=build /src/build /app
```

The `marker` simulates the Maven `~/.m2` repository: each build adds a line to it.

```
podman build --no-cache -f Dockerfile.cache -t cache-demo . 2>&1 | grep dep-
podman build --no-cache -f Dockerfile.cache -t cache-demo . 2>&1 | grep dep-
```

Observe one `dep-...` line at the first build, **two** at the second — even though `--no-cache` rebuilt everything. The `/root/.m2` folder survived between the two builds.

```
podman run --rm cache-demo ls /root/.m2
```

Observe `ls: /root/.m2: No such file or directory`: the cache is **not** in the image.

Explanation. The *cache mount* is a folder held by Buildah in your user storage, mounted into the build container for the duration of one instruction. On a real project, `RUN --mount=type=cache,target=/root/.m2 mvn package` avoids re-downloading 300 MB of dependencies at every build — even when `pom.xml` changes. No `# syntax=` line was needed: Buildah understands `--mount` natively.

Step 6 — musl or glibc?

```
podman run --rm --entrypoint sh docker.io/library/eclipse-temurin:21-jre-alpine -c 'ldd --
version 2>&1 | head -1; head -1 /etc/os-release'
podman run --rm --entrypoint sh docker.io/library/eclipse-temurin:21-jre -c 'ldd --version 2>&1
| head -1; head -1 /etc/os-release'
```

Observe `musl libc (x86_64)` / `Alpine Linux` on one side, `ldd (Ubuntu GLIBC 2.xx)` / `Ubuntu` on the other.

```
podman run --rm --entrypoint sh docker.io/library/eclipse-temurin:21-jre-alpine -c 'apk info |
wc -l'
podman run --rm --entrypoint sh docker.io/library/eclipse-temurin:21-jre -c 'dpkg -l | grep -c
^ii'
```

Observe about `73` packages against `140`.

Explanation. That is the command to run **before** migrating an image to Alpine: if a native library of your application was compiled for `glibc`, it will not load with `musl`. The number of packages, on the other hand, is what a vulnerability scanner counts: half the packages, half the potential CVEs.

Step 7 — Distroless: no shell at all

```
podman pull gcr.io/distroless/java21-debian12
podman images --format '{{.Repository}} {{.Size}}' | grep distroless
```

Observe `194 MB` — less than the Alpine JRE, even though it is Debian.

Create `Dockerfile.distroless`:

```
FROM gcr.io/distroless/java21-debian12
COPY --from=localhost/api-multi:1.0 /app/api.jar /app/api.jar
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
```

```
podman build -f Dockerfile.distroless -t api-distroless:1.0 .
podman run -d --name d -p 18082:8080 api-distroless:1.0
sleep 2 ; curl -s localhost:18082/actuator/health ; echo
podman exec d sh -c 'ls'
podman exec d id
podman rm -f -t 0 d
```

Observe `{"status":"UP"}` — the API runs — then `executable file sh` not found in `$PATH` and the same error for `id`: there is **nothing** in this image besides Java and your JAR.

Explanation. `COPY --from=` also accepts the name of an existing **image**, not just a stage. And an image without a shell is an image where an attacker who obtains code execution finds neither `sh`, nor `curl`, nor `apt` — but where you cannot get in either. You compensate with rich logs, `/actuator`, and `podman cp` to extract a file.

Clean-up

```
podman rmi api-mono:1.0 api-multi:1.0 web-multi:1.0 cache-demo api-distroless:1.0 \
    gcr.io/distroless/java21-debian12 docker.io/library/node:22-alpine 2>/dev/null
podman rmi $(podman images --filter dangling=true -q) 2>/dev/null
podman images --format '{{.Repository}}:{{.Tag}}'
```

Observe that `alpine`, `nginx:alpine`, `eclipse-temurin:21-jdk`, `eclipse-temurin:21-jre` and `eclipse-temurin:21-jre-alpine` remain. Keep `~/labo-docker/05/Dockerfile`: the stack of the following labs will use it.

What you must be able to state now

- A single-stage image weighs as much as its tooling: 488 MB for a 2 KB JAR.

- Multi-stage keeps only the copied artefact: 209 MB, without sources or compiler — and `--target` lets you inspect a stage.
- The Angular front end in production is a 64 MB nginx image; Node is not in it.
- A *cache mount* survives builds and does not enter the image; Buildah handles it without `#syntax=`.
- Alpine = `musl`, Ubuntu = `glibc`: `ldd --version` says so, and a test validates it.
- Distroless: `{"status":"UP"}` but no `sh` — security versus observability.